# WEEK 4 | GUI & Problem-Solving Agent

**Lab Objective**

The objective of this lab is to:

- Build a simple **Graphical User Interface (GUI)** using **Streamlit**.
- Understand how a **Problem-Solving Agent** works in practice
- Implement the agent's reasoning steps in a **structured computational pipeline**

**Tools & Technologies**

- Python 3.x
- Streamlit
- Standard Python libraries (lists, dictionaries, functions)

## Part A: GUI Development Using Streamlit

**Purpose**

Streamlit allows rapid development of interactive web applications using pure Python.

***Official Documentation:*** *https://docs.streamlit.io/*

**Basic Streamlit Workflow**

A Streamlit application follows a **top-to-bottom execution model**:

1. Define page title and description
2. Accept user inputs through widgets
3. Process inputs using backend logic
4. Display outputs dynamically

**Commonly Used Streamlit Components**

| Component | Usage |
|---|---|
| st.title() | Display application title |
| st.text_input() | Accept textual input |
| st.number_input() | Accept numeric input |
| st.selectbox() | Select one option from multiple |

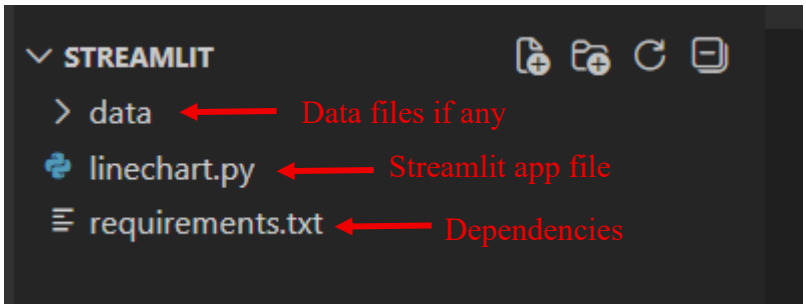
| | |
|---|---|
| st.button() | Trigger computation |
| st.write() | Display text or results |
| st.success() | Display success messages |

## Setup Instructions

### 1. Create Your Python File

- Open **VS Code** (not Jupyter Notebook)
- Create a new file with .py extension
- **Correct:** app.py, interface.py, dashboard.py
- **Incorrect:** .ipynb, .txt, or any other format

### 2. File Structure

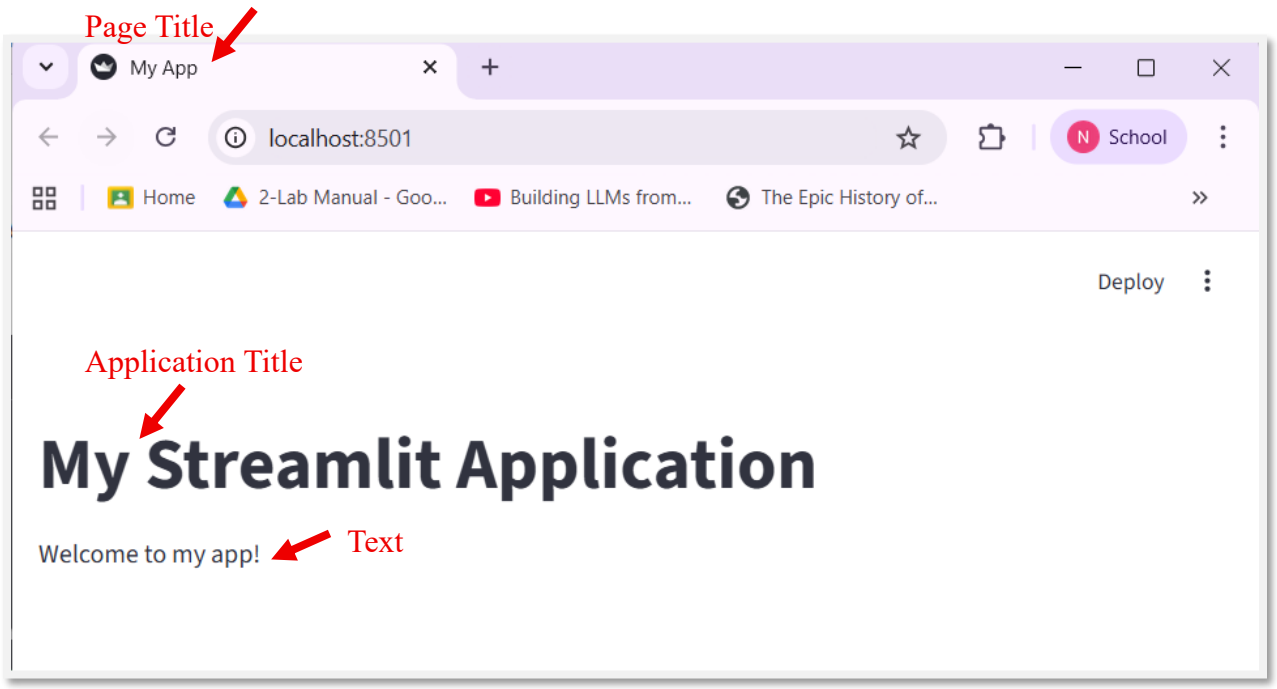



### 3. Basic Streamlit Template

Start with this minimal template in your .py file:

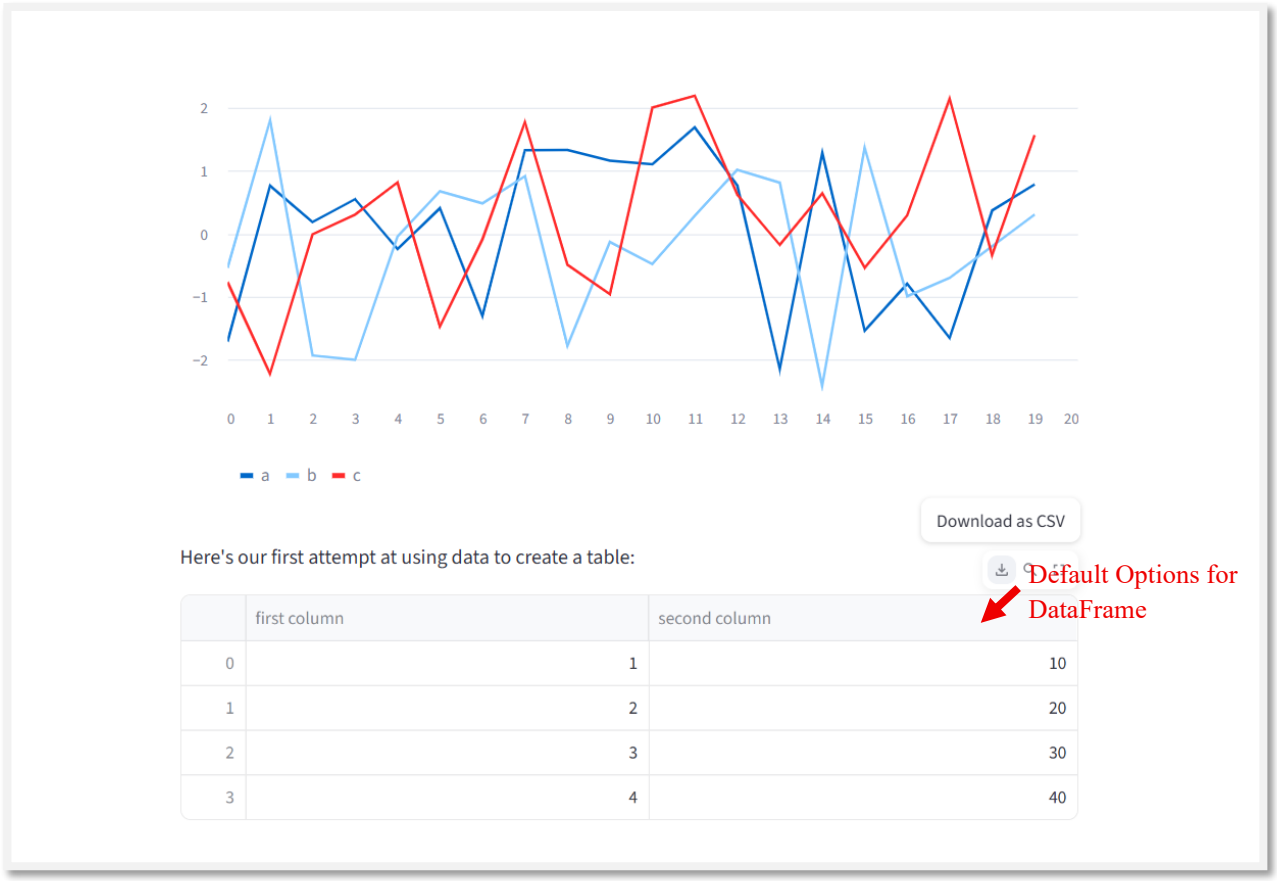
```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt

# Set page configuration (first command)
st.set_page_config(page_title="My App", layout="wide")

# application code
st.title("My Streamlit Application")
st.write("Welcome to my app!")
```

**Command to Run Streamlit web App:** streamlit run fileName.py




**Example#1 Interface:**

```
import streamlit as st
import numpy as np
import pandas as pd

# Line Chart

chart_data = pd.DataFrame(
     np.random.randn(20, 3),
     columns=['a', 'b', 'c'])

st.line_chart(chart_data)

# Dataframe Display

st.write("Here's our first attempt at using data to create a table:")
st.write(pd.DataFrame({
    'first column': [1, 2, 3, 4],
    'second column': [10, 20, 30, 40]
}))
```




**Example#2 Interface:**

```
import streamlit as st

st.title("AI Input-Output Demo")

number = st.number_input("Enter a number", value=0)

if st.button("Calculate Square"):
    st.success(f"Model Output: {number ** 2}")
```

**Example#3: Markdown**

**Basic Syntax**

```
st.markdown("<p>Hello</p>", unsafe_allow_html=True)
```

You can add **styling**

**Change Font Size**

```
st.markdown(
    "<p style='font-size:24px;'>Hello World</p>",
    unsafe_allow_html=True
)
```

**Change Font Color**

```
st.markdown(
    "<p style='color:red;'>Hello World</p>",
    unsafe_allow_html=True
)
```

**Change BOTH Size and Color**

```
st.markdown(
    "<p style='font-size:28px; color:blue;'>Styled Text</p>",
    unsafe_allow_html=True
)
```

**Two buttons, CSS applied to only one**

A simple reliable approach is to use HTML buttons with unique IDs:

```
import streamlit as st
st.markdown("""
<style>
#red-button {
    background-color: red;
    color: white;
```

```
    padding: 10px 20px;
    border-radius: 8px;
}
</style>
""", unsafe_allow_html=True)
st.markdown('<button id="red-button">Delete</button>', unsafe_allow_html=True)
st.button("Save")
```

Here:

- **Delete** → custom CSS applied
- **Save** → normal Streamlit button

The important concept is that CSS needs a **selector** to identify *which element* to style. An id is one way to uniquely identify a specific HTML element.

**Key Points**

1. **Streamlit reruns the entire script on every interaction**
   Typing, clicking, or changing input → full rerun (by design).
2. **Heavy logic should never run by default**
   Loops, ML models, and long computations must be **inside if st.button()** or similar conditions.
3. **Use caching for expensive work**
   @st.cache_data or @st.cache_resource prevents recomputation on every rerun.
   **Example:**

```
@st.cache_data
def load_data():
    df = pd.read_csv("students.csv")
    return df
data = load_data()
```

```
@st.cache_resource
def load_model():
    model = joblib.load("model.pkl")
    return model
prediction = model.predict([[25, 50000]])
```

- **@st.cache_data** is for caching **data/results**, such as DataFrames, API responses, or calculation results. Streamlit manages these as cached data and returns copies.
- **@st.cache_resource** is for caching **long-lived resources/objects**, such as ML models, database connections, or large model pipelines. Streamlit keeps and reuses the same object.

4. **Rerun is cheap, computation is not**
   UI reruns are fast; only uncontrolled loops make apps slow.

| Concept | Purpose | Example Use Case |
|---|---|---|
| Heavy logic should never run by default | Control when code runs | Button-gated ML inference, loops, or dataset processing |
| Caching expensive work | Control how many times code runs | Large dataset preprocessing, ML model predictions, repeated computations |

***For more examples, Explore Official Documentation:*** *https://docs.streamlit.io/*

## Part B: Problem Solving Agent Architecture

### State Representation

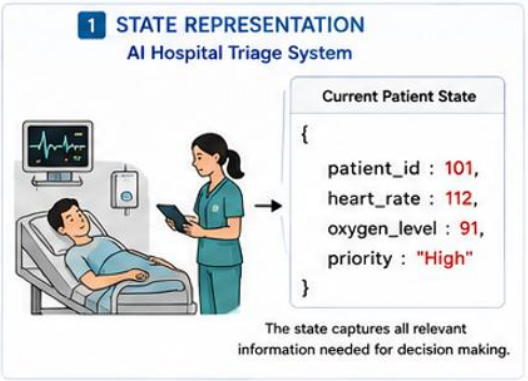
A state represents the complete information required by an intelligent agent to describe its current situation at a specific point in time.

- States may be represented using dictionaries, tuples, strings, or custom objects.
- The representation should capture all relevant information needed for decision making.
- Well-designed state representations simplify reasoning and state transitions.

### Use Case: AI Hospital Triage System

Represent the current condition of a patient before making any medical decision.

```
# Current patient state
patient_state = {
    "patient_id": 101,
    "heart_rate": 112,
    "oxygen_level": 91,
    "priority": "High"
}

print(patient_state)
```




**Tip:**

- Store all relevant attributes within the state.

- Choose immutable structures (e.g., tuples) when states need to be stored inside sets or dictionaries

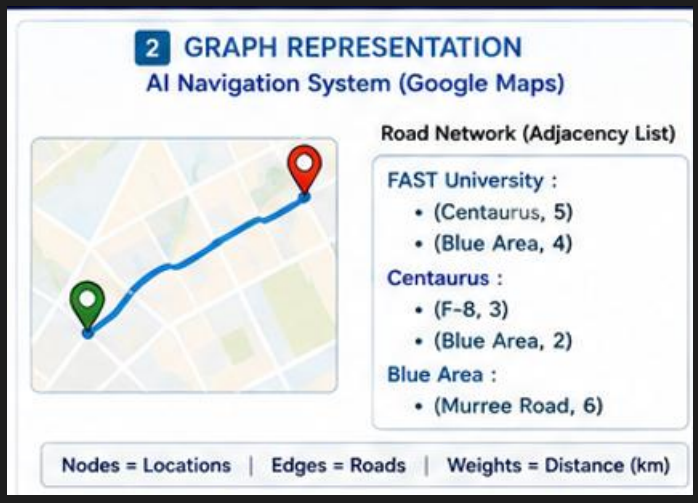
**Representing Graphs or Adjacency Lists**

Many AI applications represent environments as graphs where each node represents a **state** and edges represent possible **transitions**.

- **Dictionaries** provide an efficient way to represent graph structures.
- Edge weights can represent distance, travel time, or cost.

**Use Case: AI Navigation System (Google Maps)**

```
road_network = {

    "FAST University": [
        ("Centaurus", 5),
        ("Blue Area", 4)
    ],

    "Centaurus": [
        ("F-8", 3),
        ("Blue Area", 2)
    ],

    "Blue Area": [
        ("Murree Road", 6)
    ]

}

for neighbor, distance in road_network["FAST University"]:
    print(neighbor, distance)
```



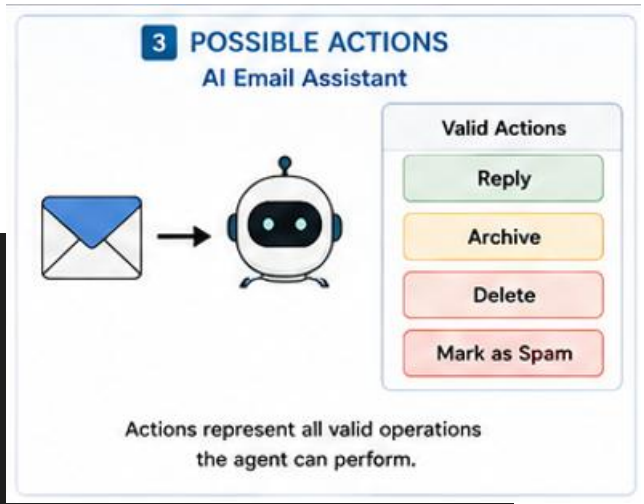

**Tip:** Store neighboring locations together with edge costs to support future search algorithms.

**Possible Actions**

Actions describe all valid operations that can transform one state into another.

- Every action modifies the current state.
- Invalid actions should not be executed.

**Use Case: AI Email Assistant**



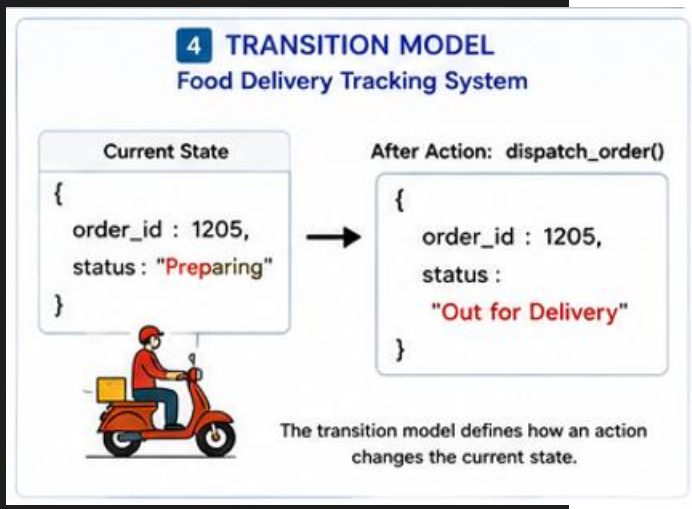

```
actions = [
    "Reply",
    "Archive",
    "Delete",
    "Mark as Spam"
]
print(actions)
```

**Tip:** Clearly define all valid actions before designing the agent's decision-making logic.

**Transition Model (Applying Actions)**

The Transition Model defines how an action changes the current state into a new state.

**Use Case: Food Delivery Tracking System**




```
def dispatch_order(order):

    order["status"] = "Out for Delivery"

    return order


order = {

    "order_id": 1205,

    "status": "Preparing"

}

updated_order = dispatch_order(order)

print(updated_order)
```

**Tip:** Always validate whether an action is applicable before updating the current state.

**Goal Test**

A Goal Test determines whether the agent has successfully achieved the desired objective.

**Use Case: Warehouse Delivery Robot**


```
def goal_test(current_location, goal_location):

    return current_location == goal_location
```

```
current_location = "Loading Dock"

goal_location = "Loading Dock"

print(goal_test(current_location, goal_location))
```

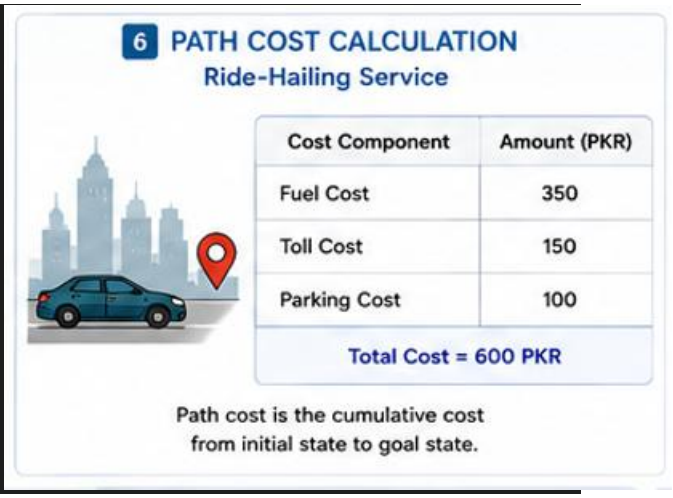
**Tip:** Keep goal tests simple and efficient because they may be evaluated repeatedly during problem solving.

**Path Cost Calculation**

Path cost represents the cumulative cost incurred while moving from the initial state to the goal state.

**Use Case: Ride-Hailing Service**

```
trip_cost = {

    "fuel": 350,

    "toll": 150,

    "parking": 100

}

total_cost = sum(trip_cost.values())

print("Total Cost:", total_cost)
```




**Tip:** The cost function may represent distance, travel time, fuel consumption, monetary expense, or any other optimization criterion.

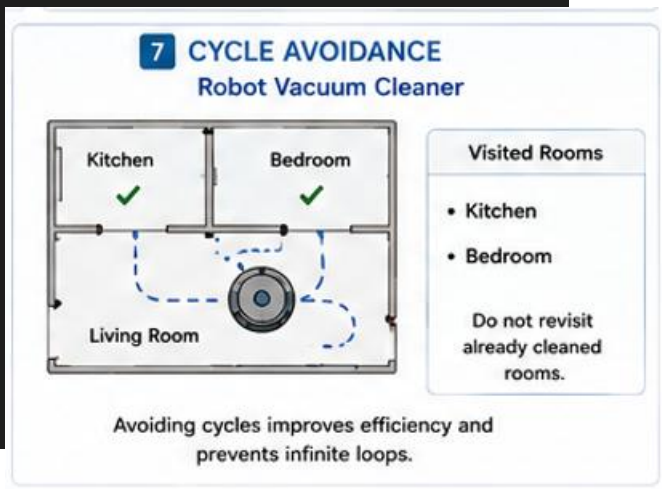
**Cycle Avoidance / Visited States**

Many AI problems contain repeated states. Revisiting previously explored states increases computation and may create infinite loops.

**Use Case: Robot Vacuum Cleaner**

```
visited_rooms = set()

rooms = [
```

```
    "Kitchen",
    "Bedroom",
    "Kitchen",
    "Living Room"
]
for room in rooms:
    if room not in visited_rooms:
        print("Cleaning:", room)
        visited_rooms.add(room)
```



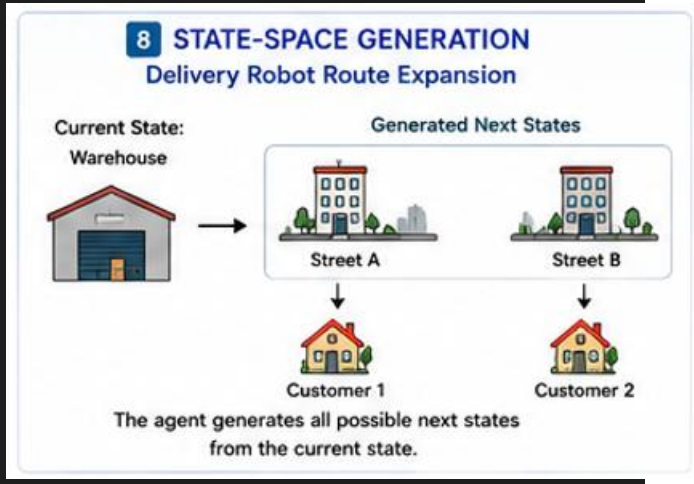

**Tip:** Sets provide fast membership testing and are commonly used to store visited states.

**State-Space Generation (Enumeration)**

State-space generation explores all possible future states reachable from the current state.

**Use Case: Delivery Robot**

```
locations = {
    "Warehouse": ["Street A", "Street B"],
    "Street A": ["Customer 1"],
    "Street B": ["Customer 2"]
}
stack = ["Warehouse"]
while stack:
    current = stack.pop()
    print(current)
    if current in locations:
```



```
        stack.extend(locations[current])
```

**Tip:** Generate the state space gradually to verify correctness before implementing search algorithms (Use **stack or queue** for explicit exploration (without BFS/DFS)).

A **Problem-Solving Agent** solves a problem by following a fixed sequence of reasoning steps. Each step converts raw input into a structured decision.

**Goal Formulation**

Goal Formulation determines what the agent intends to achieve based on the current situation and the performance measure.

**Use Case: Smart Home Climate Control**

```
def formulate_goal(current_temperature):
    if current_temperature > 28:
        return "Cool Room"
    elif current_temperature < 20:
        return "Heat Room"
    else:
        return "Maintain Temperature"

goal = formulate_goal(31)
print(goal)
```

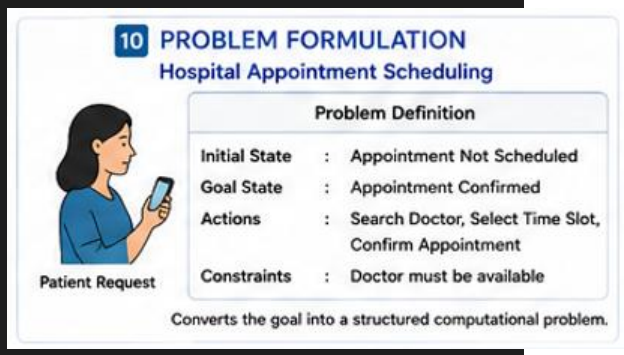

**Tip:** A well-defined goal guides all subsequent decision-making performed by the agent.

**Problem Formulation**

Problem Formulation converts the selected goal into a structured computational problem by defining the initial state, goal state, actions, and constraints.

**Use Case: Hospital Appointment Scheduling**

```
problem = {

    "initial_state": "Appointment Not Scheduled",

    "goal_state": "Appointment Confirmed",

    "actions": [

        "Search Doctor",

        "Select Time Slot",

        "Confirm Appointment"

    ],

    "constraints": "Doctor must be available"

}

print(problem)
```




**Tip:** Clearly defining the problem reduces ambiguity during the search process.

**Searching for a Solution**

Searching systematically explores available actions until a sequence leading to the goal state is discovered.

**Use Case: Hotel Booking Assistant**

```
hotels = [

    {"name": "Hotel A", "available": False},

    {"name": "Hotel B", "available": False},

    {"name": "Hotel C", "available": True}

]

for hotel in hotels:
```

```
    if hotel["available"]:
        print("Selected:", hotel["name"])
        break
```

**Tip:** Searching is the process of evaluating possible alternatives until a satisfactory solution is identified.

**Executing the Solution**

After identifying a valid solution, the agent executes each action sequentially until the desired goal is achieved.

**Use Case: Food Delivery Workflow**

```
delivery_steps = [
    "Accept Order",
    "Assign Rider",
    "Pick Up Food",
    "Deliver Order"
]

for step in delivery_steps:
    print(step)

print("Delivery Completed Successfully")
```


**Tip:** Execution applies the selected plan in the real environment and produces the final outcome.

**Complete Agent Flow (Conceptual)**

User Input/Request (Raw) → Goal Formulation

→ Problem Formulation

→ Search for Solution

→ Solution/pipeline Execution

→ Final Output

**Use Case:** AI Hospital Appointment Assistant